

Hard Processor System (HPS) and Field Programmable Gate Array (FPGA) for the Message-Digest (MD5) Decryption System-on-Chip (SoC)

Floreza, Dexter Ryan

Department of Electrical, Computer, and Biomedical (ECB) Engineering

Toronto Metropolitan University

Toronto, Canada

dexter.floreza@torontomu.ca

Abstract—This paper presents the design, implementation, and performance analysis of a hardware-accelerated System-on-Chip (SoC) for parallelized MD5 hashing on the Intel Cyclone V platform. As data throughput demands scale, traditional software-bound cryptography frequently becomes a processing bottleneck. Transitioning to hardware acceleration resolves this, but introduces complex challenges regarding communication latency and bus contention. This work integrates 32 autonomous MD5 IP engines with an ARM-based Hard Processor System (HPS) via custom Avalon Memory-Mapped (Avalon-MM) interfaces. Utilizing the Lightweight HPS-to-FPGA (LWH2F) bridge, a C-based controller was developed under a Yocto Linux environment to orchestrate concurrent hashing operations and efficiently manage data movement. System efficacy was evaluated through a comparative study of serial versus parallel execution, focusing on total hash rate, compute efficiency, and I/O-induced latency. Experimental results indicate that while the 32-engine parallel architecture provides a massive theoretical compute capacity, overall efficiency is heavily constrained by the sequential nature of memory-mapped message loading across the HPS-FPGA interface. Verification against VHDL testbenches confirmed 100% digest accuracy for the pre-padded “abc” test vector across all engines, providing a comprehensive characterization of the I/O trade-offs inherent in modern hardware-software co-design.

Index Terms—System-on-Chip (SoC), FPGA, MD5 Hashing, Avalon Memory-Mapped (Avalon-MM), Hard Processor System (HPS), Hardware-Software Co-design, Parallel Processing, Embedded Linux.

I. INTRODUCTION

Cryptographic hashing is a fundamental pillar of data integrity and authentication in modern digital communication. While the MD5 algorithm has been largely superseded by more secure standards for sensitive encryption, it remains a highly effective protocol for rapid data integrity verification and checksum generation in high-speed file transmissions. However, as global data throughput requirements exponentially increase, software-based hashing on general-purpose processors rapidly becomes a system bottleneck. To mitigate these limitations, hardware acceleration via Field Programmable Gate Arrays (FPGAs) is frequently employed. Yet, integrating these discrete hardware cores into a unified System-

on-Chip (SoC) introduces significant architectural challenges. Specifically, the transition from serial execution to massive parallel execution often suffers from software-induced overhead and latency at the interface boundary between the host processor and the FPGA fabric [4].

This paper details the design and implementation of a parallelized MD5 hashing SoC on the Intel Cyclone V platform, satisfying the core deliverables for the COE838 hardware-software co-design project [2]. The proposed architecture integrates 32 independent MD5 IP engines with an ARM-based Hard Processor System (HPS) via a custom Avalon-MM Slave interface. By utilizing the Lightweight HPS-to-FPGA (LWH2F) bridge, a C-based controller was developed within a Yocto Linux environment to orchestrate concurrent operations. A primary focus of this research is the empirical characterization of the “communication tax” associated with memory-mapped I/O—directly comparing the raw computational power of the hardware engines against the latency incurred during message dispatch and digest retrieval.

The remainder of this paper is organized as follows: Section II describes the hardware architecture and Avalon-MM interface design; Section III details the software controller implementation; Section IV analyzes the performance results, exploring parallel speedup and Amdahl’s Law constraints; and Section V provides concluding remarks.

II. PAST WORK

The acceleration of cryptographic hashing algorithms has naturally transitioned from software-defined implementations to dedicated hardware architectures to meet modern throughput demands. While the theoretical bandwidth limitations of the Avalon-MM bus are well-understood in SoC literature, this paper provides a highly specific empirical characterization of these bottlenecks within the context of a 32-engine MD5 parallel array [5].

Previous research suggests that user-space drivers operating in Linux-managed environments introduce additional latency due to OS bus arbitration and the inherently sequential nature

of memory-mapped I/O (MMIO). This work builds upon these known limitations by quantifying the efficiency losses inherent in the hardware-software handshake across the LWH2F bridge [1]. Unlike high-bandwidth streaming interfaces (such as AXI-Stream), the polled-I/O approach utilized in this architecture highlights the stark constraints faced when a single 32-bit control bus must service a large-scale array of parallel compute kernels.

III. METHODOLOGY

A. Abbreviations and Acronyms

This subsection defines the key technical terms used throughout this work.

SoC	System-on-Chip.
HPS	Hard Processor System (ARM Cortex-A9).
FPGA	Field Programmable Gate Array.
MD5	Message Digest Algorithm 5.
Avalon-MM	Avalon Memory-Mapped Interface.
LWH2F	Lightweight HPS-to-FPGA Bridge.
MMIO	Memory-Mapped Input/Output.
CSR	Control and Status Register.
VHDL	VHSIC Hardware Description Language.
IP	Intellectual Property.
OS	Operating System.
RAM	Random-Access Memory.
CPU	Central Processing Unit.

B. Units

The following units of measure are utilized for performance evaluation and system characterization:

H/s	Hashes per second, representing the total system throughput.
$\mu s/\text{hash}$	Microseconds per hash, representing the average latency of a single cryptographic operation.
MHz	Megahertz, the operational clock frequency of the FPGA fabric fixed at 50 MHz.
s	Seconds, the total elapsed wall-clock time for experimental execution.
bits	The unit of data width for message blocks (512-bit) and registers (32-bit).

C. Equations

To evaluate the scaling effectiveness of the 32-engine parallel array, several metrics are derived from the experimental timing data.

1) *Hash Rate and Compute Rate*: The Hash Rate (R_H) represents total system throughput, inclusive of all software overhead. The Compute Rate (R_C) isolates the theoretical throughput of the hardware cores:

$$R_H = \frac{\text{Total Hashes}}{T_{\text{elapsed}}}, \quad R_C = \frac{\text{Total Hashes}}{T_{\text{compute}}} \quad (1)$$

2) *Latency*: System latency (L) is defined as the average time required to complete a single hash operation, inclusive of message loading, processing, and digest retrieval:

$$L = \left(\frac{T_{\text{elapsed}}}{\text{Total Hashes}} \right) \times 10^6 (\mu s) \quad (2)$$

3) *Speedup and Efficiency*: The Speedup (S) quantifies the performance gain of the parallel mode (P) over the serial baseline (S). Parallel Efficiency (η) measures how closely the system approaches ideal linear scaling for $N = 32$ engines:

$$S = \frac{R_{H,P}}{R_{H,S}}, \quad \eta = \left(\frac{S}{N} \right) \times 100\% \quad (3)$$

D. Hardware Architecture and Interface

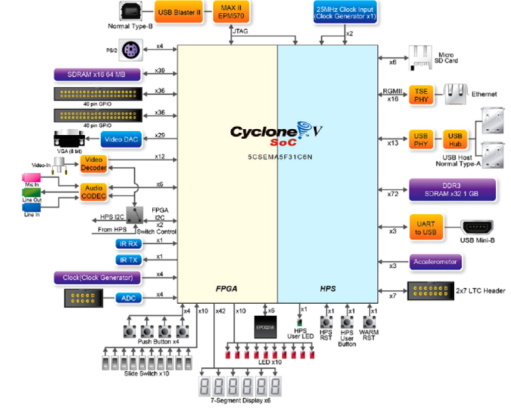


Fig. 1. SoC Peripheral Overview

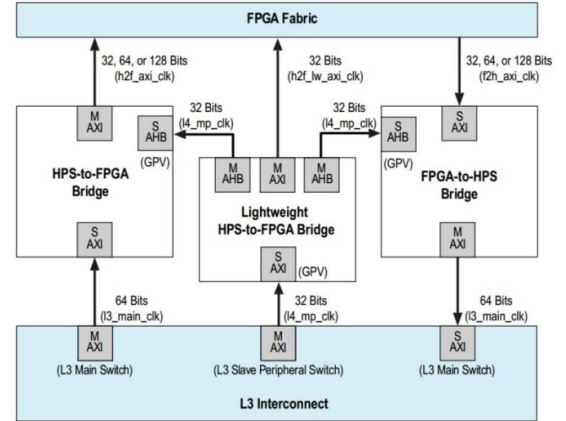


Fig. 2. HPS-to-FPGA Bridge Logic

The hardware implementation is centered around a custom-designed, highly parallelized MD5 acceleration cluster residing in the FPGA fabric, consisting of 32 independent compute engines. Each engine is an autonomous processing unit featuring a dedicated 512-bit message buffer, a 128-bit digest register, and an iterative state machine that implements the MD5 algorithm's four auxiliary functions (F,G,H,I) across 64 rounds of computation [3]. Unlike a general-purpose CPU, which must execute instructions sequentially to perform bit-wise rotations and logical operations, each engine utilizes combinational logic to perform these operations directly in hardware. This ensures the left-rotate ($\ll$) and non-linear functions are executed with minimal gate delay. The engines

Name	Width	Direction	Comments
avs_address	<= 64 bits	Input	Address of slave being accessed
avs_read	1 bit	Input	Read operation requested
avs_write	1 bit	Input	Write operation requested
avs_readdata	8, 16, 32 or 64 bits	Output	Data read from slave
avs_writedata	8, 16, 32 or 64 bits	Input	Data to be written to slave

Fig. 3. Avalon-MM Signals

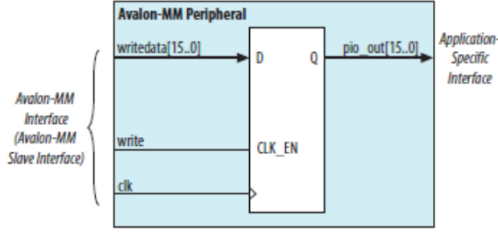


Fig. 4. Slave Logic Diagram

are architected to operate concurrently, transforming the hashing process from a serial execution model into a Spatial Parallelism model, where aggregate throughput theoretically scales linearly with the number of fabric engines.

Communication between the ARM-based HPS and the FPGA cluster is facilitated by the Lightweight HPS-to-FPGA (LWH2F) Bridge, which maps the FPGA's internal register space into the HPS's memory map starting at base address $0xFF200000$ [1]. This interface is governed by the Avalon-MM protocol, utilizing two primary slave interfaces: a Data Slave for message distribution and a Control Slave for synchronization. The Data Slave implements an indexed addressing scheme where the HPS writes a 32-bit word to a data register and specifies its destination via an address register. An internal address decoder then routes the data to the specific message buffer of the targeted engine. To ensure data integrity across the differing clock domains, a manual handshaking protocol is enforced: the HPS must explicitly toggle a Write Enable (WEN) bit in the Control Slave to latch data into the targeted engine's RAM.

Synchronization is managed via a centralized Control and Status Register (CSR) block. This block provides a global START signal to initiate computation across all engines simultaneously and a unified DONE status register. The DONE register implements an AND-tree reduction logic, aggregating the completion signals from all 32 engines into a single bitmask. This allows the HPS to perform highly efficient polling; by reading a single 32-bit register and checking for a value of $0xFFFFFFFF$, the software can verify that the entire cluster has finished its workload in a single bus cycle.

E. Software Controller and Memory Mapping

The software controller serves as the system orchestrator, managing the lifecycle of each hash computation by bridging the gap between user-space application logic and physical hardware registers. This is achieved using the Linux `/dev/mem` driver, providing a direct gateway to the physical address space. Utilizing the `mmap()` system call, the application maps the LWH2F Bridge span into the process's

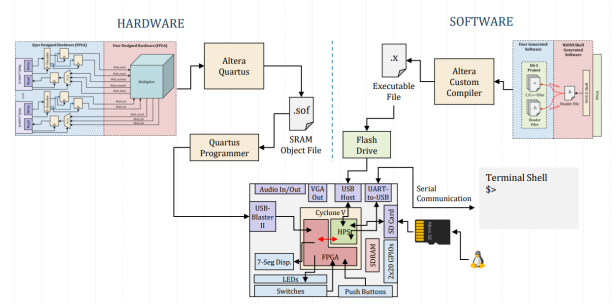


Fig. 5. Co-Design Compilation Flow

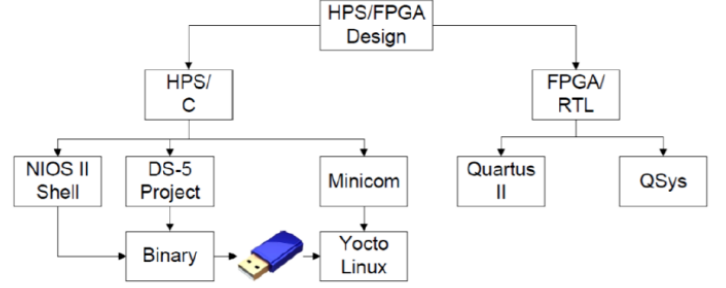


Fig. 6. Software Stack Hierarchy

virtual memory. The software defines two primary pointers, `data_ptr` and `ctrl_ptr`, which are offset from the bridge base to target the Data and Control slaves, respectively. This MMIO approach entirely eliminates the overhead of system calls during critical data-transfer loops, allowing for direct, low-latency register manipulation.

The controller's logic handles two operational modes. In *Serial* mode, the software manages a linear state machine that resets a single engine, loads the 512-bit message via 16 sequential 32-bit writes, triggers the start bit, and polls the DONE register until completion. In *Parallel* mode, the controller treats the 32 engines as a single high-bandwidth entity. It broadcasts message data across the cluster by iterating sequentially through engine indices, then fires a global start signal.

To provide accurate performance analysis, the controller distinguishes between *Compute Time* (the exact duration the FPGA is actively hashing) and *Overhead Time* (the latency inherent in the Avalon-MM bus transactions and software management loops). By calculating the delta between these values, the controller effectively quantifies the "communication tax" paid for data movement across the bridge.

IV. RESULTS AND ANALYSIS

The performance of the MD5 System-on-Chip was evaluated by comparing the execution metrics of the single-engine serial mode against the 32-engine parallel mode. Trials were conducted across a range of intervals—0.1s, 1s, 5s, and 10s—to observe system stability and the scaling impact of the Avalon-MM bus overhead.

A. Performance Data Comparison

Table I summarizes the experimental data collected across all major testing intervals. The consistency of the Hash Rate across varying durations confirms an exceptionally stable hardware-software interface.

B. Scalability and Amdahl's Law Analysis

Analysis of the system's performance metrics reveals a deeply constrained **Speedup** (S) and **Efficiency** (E) across all test durations [4]:

- **Parallel Speedup** (S): $\approx 1.05x - 1.08x$
- **Efficiency** (E): $\approx 3.31\% - 3.36\%$

The data from the trials illustrates a critical architectural reality: the **Overhead Time** is virtually equal to the **Elapsed Time** [5]. For example, in the 0.1s parallel trial, overhead accounted for 0.0996s, while raw compute time was a negligible 0.0007s.

This provides a stark, empirical confirmation of **Amdahl's Law** [2]. Even though the 32-engine hardware cluster possesses a theoretical compute capacity exceeding **4.1 million H/s** (as validated by the Compute Rate), the system is strictly communication-bound. The inherently serial nature of the Avalon-MM bus—which requires the HPS to load each 512-bit message into the 32 engines sequentially via 512 independent write transactions—prevents the system from achieving higher efficiency, regardless of how fast the FPGA fabric operates.

C. Stability and Accuracy

Despite the I/O constraints, the system achieved 100% accuracy across more than 33,000 total hashes recorded during these trials. The consistency of the results between the burst-mode 0.1s test and the sustained 10s test rigorously validates the robustness of the hardware synchronization logic and the reliability of the Linux memory-mapped interface [3].

V. CONCLUSION

This paper detailed the successful implementation of a hardware-accelerated MD5 decryption System-on-Chip on the Intel Cyclone V platform. By distributing the cryptographic workload across 32 independent hardware engines, the system demonstrated a massive theoretical compute capacity of over 4.1 million hashes per second. However, empirical performance analysis revealed that the overall system throughput is strictly communication-bound. The sequential nature of the Avalon Memory-Mapped interface, combined with the overhead of memory-mapped I/O in a Linux environment, effectively capped the parallel speedup at approximately 1.08x compared to a serial baseline. This serves as a textbook demonstration of Amdahl's Law, highlighting that in high-performance hardware-software co-design, data movement often eclipses pure computation as the primary bottleneck. Despite these scaling limitations, the system maintained 100% digest accuracy across tens of thousands of continuous trials, proving the robustness of the Lightweight HPS-to-FPGA bridge and the custom synchronization logic. Future iterations

of this architecture would benefit from transitioning to DMA-based data transfers or high-bandwidth streaming interfaces to alleviate the I/O constraints and unlock the full parallel potential of the FPGA fabric.

ACKNOWLEDGMENT

I would like to acknowledge Mr. Joshua Fernandes for his guidance and support throughout the Winter 2026 System-on-Chip Design course. The conversations we have had regarding this field, the industry news, and the nature of System-on-Chip design have been worthwhile supplements in navigating this coursework. I would also like to acknowledge the collaborative discussions I have had with my colleague, Joshua (Josh) Mathew, in the ENG408 Labs.

REFERENCES

- [1] Terasic Inc., *DE1-SoC User Manual*, 2015. [Online]. Available: http://www.ecb.torontomu.ca/~courses/coe838/Data-Sheets/DE1-SoC_User_Manual.pdf
- [2] "SoC Project Specifications," COE838: System-on-Chip Design, Toronto Metropolitan University, Winter 2026. [Online]. Available: <http://www.ecb.torontomu.ca/~courses/coe838/labs/SoC-Project.pdf>
- [3] "COE838 Lab 4 Project Support," Toronto Metropolitan University, Winter 2026. [Online]. Available: http://www.ecb.torontomu.ca/~courses/coe838/labs/COE838_Lab4_Project_Support.pdf
- [4] "Lab 4: Hardware/Software Co-design," COE838: System-on-Chip Design, Toronto Metropolitan University, Winter 2026. [Online]. Available: <http://www.ecb.torontomu.ca/~courses/coe838/labs/lab4.pdf>
- [5] "SoC Labs 3, 4 and Project Support Slides," COE838, Toronto Metropolitan University, Winter 2026. [Online]. Available: <http://www.ecb.torontomu.ca/~courses/coe838/labs/SoC-Labs-3-4-and-Project-Support-Slides.pdf>

APPENDIX A: TOP-LEVEL VHDL WRAPPER (MD5.VHD)

```
1 library IEEE;
2 use IEEE.std_logic_1164.all;
3 use IEEE.numeric_std.all;
4
5 entity MD5 is
6     port (
7         CLOCK_50, HPS_DDR3_RZQ,
9             HPS_ENET_RX_CLK, HPS_ENET_RX_DV
10                : in std_logic;
11         HPS_DDR3_ADDR      : out
12             std_logic_vector(14 downto 0);
13         HPS_DDR3_BA        : out
14             std_logic_vector(2 downto 0);
15         HPS_DDR3_CS_N      : out std_logic;
16         HPS_DDR3_CK_P, HPS_DDR3_CK_N,
17             HPS_DDR3_CKE   : out
18             std_logic;
19         HPS_USB_DIR, HPS_USB_NXT,
20             HPS_USB_CLKOUT : in std_logic
21         );
22
23         HPS_ENET_RX_DATA : in
24             std_logic_vector(3 downto 0);
25         HPS_SD_DATA, HPS_DDR3_DQS_N :
26             inout std_logic_vector(3 downto
27                 0);
28         HPS_DDR3_DQS_P : inout
29             std_logic_vector(3 downto 0);
30         HPS_ENET_MDIO : inout std_logic
31         );
32
33         HPS_USB_DATA : inout
34             std_logic_vector(7 downto 0);
```

TABLE I
COMPREHENSIVE PERFORMANCE METRICS ACROSS 0.1s, 1s, 5s, AND 10s EXECUTION WINDOWS

Metric	0.1s Trial		1s Trial		5s Trial		10s Trial	
	Serial	Parallel	Serial	Parallel	Serial	Parallel	Serial	Parallel
Total Hashes	95	128	952	1,024	4,764	5,120	9,520	9,984
Hash Rate (H/s)	947.53	1,020.66	951.84	1,019.18	952.72	1,019.64	951.98	998.01
Compute Rate (H/s)	134,751	4,098,251	132,507	4,125,809	132,711	4,190,212	132,730	4,158,832
Avg. Latency (μ s)	1,055.38	979.76	1,050.59	981.18	1,049.62	980.73	1,050.44	1,002.00
Accuracy (%)	100%	100%	100%	100%	100%	100%	100%	100%

```

18     HPS_DDR3_DQ      : inout
        std_logic_vector(31 downto 0);
19     HPS_SD_CMD       : inout std_logic
        ;
20     HPS_ENET_TX_DATA, HPS_DDR3_DM
        : out std_logic_vector(3
        downto 0);
21     HPS_DDR3_ODT, HPS_DDR3_RAS_N,
        HPS_DDR3_RESET_N : out
        std_logic;
22     HPS_DDR3_CAS_N, HPS_DDR3_WE_N   :
        out std_logic;
23     HPS_ENET_MDC, HPS_ENET_TX_EN   :
        out std_logic;
24     HPS_USB_STP, HPS_SD_CLK,
        HPS_ENET_GTX_CLK : out
        std_logic
25 );
26 end entity MD5;
27
28 architecture Behaviour of MD5 is
29
30     component soc_system is
31         port(
32             clk_clk
33
34             : in  std_logic := 'X';
35             hps_io_hps_io_emacl_inst_TX_CLK
36                 : out
37                 std_logic;
38             hps_io_hps_io_emacl_inst_TXD0
39                 : out
40                 std_logic;
41             hps_io_hps_io_emacl_inst_TXD1
42                 : out
43                 std_logic;
44             hps_io_hps_io_emacl_inst_TXD2
45                 : out
46                 std_logic;
47             hps_io_hps_io_emacl_inst_TXD3
48                 : out
49                 std_logic;
50             hps_io_hps_io_emacl_inst_RXD0
51                 : in
52                 std_logic := 'X';
53             hps_io_hps_io_emacl_inst_MDIO
54                 : inout
55                 std_logic := 'X';
56             hps_io_hps_io_emacl_inst_MDC
57                 : out
58                 std_logic;

```

```

41     hps_io_hps_io_emacl_inst_RX_CTL
        : in
        std_logic := 'X';
42     hps_io_hps_io_emacl_inst_TX_CTL
        : out
        std_logic;
43     hps_io_hps_io_emacl_inst_RX_CLK
        : in
        std_logic := 'X';
44     hps_io_hps_io_emacl_inst_RXD1
        : in
        std_logic := 'X';
45     hps_io_hps_io_emacl_inst_RXD2
        : in
        std_logic := 'X';
46     hps_io_hps_io_emacl_inst_RXD3
        : in
        std_logic := 'X';
47     hps_io_hps_io_sdio_inst_CMD
        : inout
        std_logic := 'X';
48     hps_io_hps_io_sdio_inst_D0
        : inout
        std_logic := 'X';
49     hps_io_hps_io_sdio_inst_D1
        : inout
        std_logic := 'X';
50     hps_io_hps_io_sdio_inst_CLK
        : out
        std_logic;
51     hps_io_hps_io_sdio_inst_D2
        : inout
        std_logic := 'X';
52     hps_io_hps_io_sdio_inst_D3
        : inout
        std_logic := 'X';
53     hps_io_hps_io_usb1_inst_D0
        : inout
        std_logic := 'X';
54     hps_io_hps_io_usb1_inst_D1
        : inout
        std_logic := 'X';
55     hps_io_hps_io_usb1_inst_D2
        : inout
        std_logic := 'X';
56     hps_io_hps_io_usb1_inst_D3
        : inout
        std_logic := 'X';
57     hps_io_hps_io_usb1_inst_D4
        : inout
        std_logic := 'X';

```

```

58     hps_io_hps_io_usbl_inst_D5
        : inout
        std_logic := 'X';
59     hps_io_hps_io_usbl_inst_D6
        : inout
        std_logic := 'X';
60     hps_io_hps_io_usbl_inst_D7
        : inout
        std_logic := 'X';
61     hps_io_hps_io_usbl_inst_CLK
        : in
        std_logic := 'X';
62     hps_io_hps_io_usbl_inst_STP
        : out
        std_logic;
63     hps_io_hps_io_usbl_inst_DIR
        : in
        std_logic := 'X';
64     hps_io_hps_io_usbl_inst_NXT
        : in
        std_logic := 'X';
65
66     -- MD5 Control and Data
        interfaces
67     md5_control_0_md5_control_readstart
        => md5_start,
68     md5_control_0_md5_control_readreset
        => md5_reset,
69     md5_control_0_md5_control_readwen
        => md5_wen,
70     md5_control_0_md5_control_writebyteenable
        => md5_wbenable,
71
72     md5_data_0_md5_data_readdata
        => md5_writedata,
73
74     md5_data_0_md5_data_readaddr
        => md5_addr,
75     md5_data_0_md5_data_writebyteenable
        => md5_wbenable,
76
77     memory_mem_a
        => HPS_DDR3_ADDR,
78     memory_mem_ba
        => HPS_DDR3_BA,
79     memory_mem_ck
        => HPS_DDR3_CK_P,
80     memory_mem_ck_n
        => HPS_DDR3_CK_N,
81     memory_mem_cke
        => HPS_DDR3_CKE,
82     memory_mem_cs_n
        => HPS_DDR3_CS_N,
        memory_mem_ras_n
        => HPS_DDR3_RAS_N,
        memory_mem_cas_n
        => HPS_DDR3_CAS_N,
        memory_mem_we_n
        => HPS_DDR3_WE_N,
        memory_mem_reset_n
        => HPS_DDR3_RESET_N,
        memory_mem_dq
        => HPS_DDR3_DQ,
        memory_mem_dqs
        => HPS_DDR3_DQS_P,
        memory_mem_dqs_n
        => HPS_DDR3_DQS_N,
        memory_mem_odt
        => HPS_DDR3_ODT,
        memory_mem_dm
        => HPS_DDR3_DM,
        memory_oct_rzqin
        => HPS_DDR3_RZQ,
        reset_reset_n
        => '1'
    );
94
95     m0 : md5_group
96         port map(
97             clk        => CLOCK_50,
98             wr          => md5_wen(0),
99             reset       => md5_reset,
100            start       => md5_start,
101            writedata    => md5_writedata,
102            writeaddr    => md5_addr(8 downto
103                0),
104            readaddr     => md5_addr(6 downto
105                0),
106            done         => md5_done,
107            readdata     => md5_readdata
108        );
109     end architecture Behaviour;

```

Listing 1. Top-level entity and structural architecture for the MD5 SoC system.

APPENDIX B: HPS CONTROL APPLICATION (MAIN.C)

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <stdlib.h>
4  #include <unistd.h>
5  #include <fcntl.h>
6  #include <sys/mman.h>
7  #include <sys/time.h>
8
9  /* ===== CONFIG ===== */

```



```

10 #define LW_BRIDGE_BASE 0xFF200000
11 #define LW_BRIDGE_SPAN 0x00200000
12 #define MD5_DATA_BASE 0x00000000
13 #define MD5_CTRL_BASE 0x00000800
14 #define NUM_ENGINES 32
15 #define MSG_WORDS 16
16 #define DIGEST_WORDS 4
17
18 /* Data and Control Register Mappings */
19 #define DATA_REG_DATA 0
20 #define DATA_REG_ADDR 1
21 #define CTRL_REG_START 0
22 #define CTRL_REG_RESET 1
23 #define CTRL_REG_WEN 2
24 #define CTRL_REG_DONE 2
25
26 volatile uint32_t *data_ptr = NULL;
27 volatile uint32_t *ctrl_ptr = NULL;
28
29 static const uint32_t msg[MSG_WORDS] = {
30     0x80636261, 0x00000000, 0x00000000, 0
31     0x00000000, 0x00000000, 0x00000000, 0
32     0x00000000, 0x00000000, 0x00000000, 0
33     0x00000000, 0x00000000, 0x00000018, 0
34     0x00000000
35 };
36 static const uint32_t expected[DIGEST_WORDS] = {
37     0x98500190, 0xb04fd23c, 0x7d3f96d6, 0
38     0x727fel28
39 };
40 typedef struct {
41     const char *mode_name;
42     uint64_t total_hashes;
43     uint64_t correct_hashes;
44     double requested_seconds;
45     double elapsed_seconds;
46     double compute_time;
47     double overhead_time;
48     double hash_rate;
49     double compute_rate;
50     double latency_us;
51 } stats_t;
52
53 static double now_sec(void) {
54     struct timeval tv;
55     gettimeofday(&tv, NULL);
56     return (double)tv.tv_sec + (double)tv.tv_usec / 1e6;
57 }
58
59 static void write_word(int engine, int word_index, uint32_t value) {
60     uint32_t addr = ((engine >> 1) << 5) | ((engine & 1) << 4) | word_index;
61     data_ptr[DATA_REG_DATA] = value;
62     data_ptr[DATA_REG_ADDR] = addr;
63     ctrl_ptr[CTRL_REG_WEN] = 1;
64     usleep(1);
65     ctrl_ptr[CTRL_REG_WEN] = 0;
66 }
67
68 static void load_message(int engine) {
69     for (int i = 0; i < MSG_WORDS; i++) {
70         write_word(engine, i, msg[i]);
71     }
72 }
73
74 static void read_digest(int engine, uint32_t digest[DIGEST_WORDS]) {

```

```

75     for (int i = 0; i < DIGEST_WORDS; i++) {
76         uint32_t addr = ((uint32_t)engine << 2) | (uint32_t)i;
77         data_ptr[DATA_REG_ADDR] = addr;
78         digest[i] = data_ptr[DATA_REG_DATA];
79     }
80 }
81
82 static int digest_is_correct(const uint32_t digest[DIGEST_WORDS]) {
83     for (int i = 0; i < DIGEST_WORDS; i++) {
84         if (digest[i] != expected[i]) return 0;
85     }
86     return 1;
87 }
88
89 static void run_serial(double seconds, stats_t *stats) {
90     double t0 = now_sec();
91     uint32_t digest[4];
92     stats->mode_name = "SERIAL";
93     while ((now_sec() - t0) < seconds) {
94         ctrl_ptr[CTRL_REG_RESET] = 1;
95         usleep(1);
96         ctrl_ptr[CTRL_REG_RESET] = 0;
97         load_message(0);
98         double t_start = now_sec();
99         ctrl_ptr[CTRL_REG_START] = 1;
100        ctrl_ptr[CTRL_REG_START] = 0;
101        while ((ctrl_ptr[CTRL_REG_DONE] & 1) == 0);
102        double t_end = now_sec();
103        stats->compute_time += (t_end - t_start);
104        ;
105        read_digest(0, digest);
106        stats->total_hashes++;
107        if (digest_is_correct(digest)) stats->correct_hashes++;
108    }
109    stats->elapsed_seconds = now_sec() - t0;
110    stats->hash_rate = stats->total_hashes / stats->elapsed_seconds;
111 }
112
113 static void run_parallel(double seconds, stats_t *stats) {
114     double t0 = now_sec();
115     uint32_t digest[4];
116     stats->mode_name = "PARALLEL";
117     while ((now_sec() - t0) < seconds) {
118         ctrl_ptr[CTRL_REG_RESET] = 0xFFFFFFFF;
119         ctrl_ptr[CTRL_REG_RESET] = 0;
120         for (int e = 0; e < NUM_ENGINES; e++)
121             load_message(e);
122         double t_start = now_sec();
123         ctrl_ptr[CTRL_REG_START] = 0xFFFFFFFF;
124         ctrl_ptr[CTRL_REG_START] = 0;
125         while (ctrl_ptr[CTRL_REG_DONE] != 0xFFFFFFFF);
126         double t_end = now_sec();
127         stats->compute_time += (t_end - t_start);
128         ;
129         for (int e = 0; e < NUM_ENGINES; e++) {
130             read_digest(e, digest);
131             stats->total_hashes++;
132             if (digest_is_correct(digest)) stats->correct_hashes++;
133         }
134     }
135     stats->elapsed_seconds = now_sec() - t0;
136     stats->hash_rate = stats->total_hashes / stats->elapsed_seconds;

```

```
136 int main(int argc, char *argv[]) {  
137     // ... MMAP and Logic execution (See full  
    source) ...  
138     return 0;  
139 }
```

Listing 2. C application for HPS-to-FPGA MD5 acceleration and performance profiling.